# ECE 25 Electronic Circuits Project

## Code Documentation & Architecture Guide

*Multi-Sensor Health & Weather Monitoring System*

---

## Table of Contents


1 Project Overview ..... 2
  1.1 Hardware Components ..... 2
  1.2 Key Features ..... 2
2 FreeRTOS Fundamentals ..... 3
  2.1 What is FreeRTOS? ..... 3
  2.2 Core Concepts Used ..... 3
3 System Architecture ..... 4
  3.1 Task Structure ..... 4
  3.2 Data Flow Diagram ..... 4
4 Code Structure ..... 5
  4.1 Pin Definitions ..... 5
  4.2 Data Structures ..... 5
5 Task Implementations ..... 6
  5.1 Task 1: DHT Sensor Reading ..... 6
  5.2 Task 2: Pulse Sensor Reading ..... 7
  5.3 Task 3: Real-Time Clock Reading ..... 9
  5.4 Task 4: Weather API Fetching ..... 9
  5.5 Task 5: Screen Display Controller ..... 11
6 Interrupt Service Routines (ISRs) ..... 13
  6.1 Screen Change Button ISR ..... 13
  6.2 Time/Date Edit Enable Button ISR ..... 13
7 Setup and Initialization ..... 14
  7.1 Initialization Sequence ..... 14
8 Helper Functions ..... 16
  8.1 Time/Date Frame Creator ..... 16
9 Memory and Performance Considerations ..... 17
  9.1 Stack Sizes Explained ..... 17
  9.2 Queue Sizes ..... 17
  9.3 Core Pinning Strategy ..... 17
10 System Behavior Summary ..... 18
  10.1 Normal Operation Flow ..... 18
  10.2 Button Interactions ..... 18

# 1 Project Overview

This project implements a **real-time multi-sensor monitoring system** using ESP32 microcontroller with FreeRTOS. The system integrates multiple sensors and displays information on an OLED screen with the ability to fetch weather data from the internet.

## 1.1 Hardware Components

- **ESP32 Development Board** (Dual-core processor)
- **DHT11 Sensor** - Temperature and humidity measurement
- **Pulse Sensor** - Heart rate monitoring
- **SH1106 OLED Display** (128×64) - Visual output
- **Push Buttons** (2) - Screen navigation and time editing
- **WiFi Module** - Internet connectivity for weather API

## 1.2 Key Features

- Real-time clock (RTC) with time/date display
- Local temperature and humidity monitoring
- Heart rate (BPM) measurement
- Weather information from OpenWeather API
- Multi-screen interface with button navigation
- Concurrent task execution using FreeRTOS

# 2 FreeRTOS Fundamentals

Before diving into the code, let's understand the key FreeRTOS concepts used in this project.

## 2.1 What is FreeRTOS?

FreeRTOS is a **real-time operating system** for microcontrollers that allows you to run multiple tasks concurrently. Instead of writing everything in a single `loop()`, you create separate tasks that run independently.

## 2.2 Core Concepts Used

### 2.2.1 Tasks

Think of tasks as independent "mini-programs" that run concurrently. Each task has:

- Its own function
- Its own stack memory
- A priority level
- Can be pinned to a specific CPU core (ESP32 has 2 cores)

```
// Creating a task
xTaskCreatePinnedToCore(
    functionName,      // Function to run
    "Task Name",       // Name for debugging
    stackSize,         // Stack memory (bytes)
    NULL,              // Parameters to pass
    priority,          // Priority (higher = more important)
    &taskHandle,       // Handle to reference the task
    coreNumber         // Which CPU core (0 or 1)
);
```

### 2.2.2 Queues

Queues are **thread-safe** data pipes between tasks. One task puts data in, another takes it out.

```
// Create a queue that holds 5 items of DHT_sensor_data type
QueueHandle_t myQueue = xQueueCreate(5, sizeof(DHT_sensor_data));

// Send data to queue (from producer task)
xQueueSend(myQueue, &data, portMAX_DELAY);

// Receive data from queue (in consumer task)
xQueueReceive(myQueue, &buffer, timeout);
```

### 2.2.3 Semaphores

Semaphores are synchronization primitives - like a token that tasks can pass around.

- **Binary Semaphore**: Acts like a flag (taken/available)
- Used for signaling between tasks or from interrupts

### 2.2.4 Interrupts (ISR)

Hardware interrupts trigger functions immediately when events occur (like button presses).

- Must be fast and minimal
- Use `IRAM_ATTR` to keep function in fast memory
- Cannot use blocking operations

### 2.2.5 Task Delays

```
vTaskDelay(1000 / portTICK_RATE_MS);  // Delay for 1000ms
```

This **yields** the CPU to other tasks instead of blocking like `delay()`.

# 3 System Architecture

## 3.1 Task Structure

The system uses **5 concurrent tasks** running on both ESP32 cores:

| Task | Core | Priority | Purpose |
|---|---|---|---|
| `readDHT` | 1 | 1 | Reads DHT11 sensor every 1s |
| `readPulseSensor` | 1 | 1 | Monitors pulse sensor continuously |
| `readRTC` | 1 | 1 | Updates time from RTC every 1s |
| `openWeatherGet` | 1 | 1 | Fetches weather API every 5s |
| `screenDisplay` | 1 | 1 | Updates OLED display every 400ms |

## 3.2 Data Flow Diagram

```
┌──────────────┐          ┌──────────────┐          ┌──────────────┐
│   DHT11      │──────────>│Queue (DHT)   │──────────>│              │
└──────────────┘          └──────────────┘          │              │
                                                    │   Screen     │
┌──────────────┐          ┌──────────────┐          │   Display    │
│Pulse Sensor  │──────────>│Queue (Pulse)│──────────>│    Task      │
└──────────────┘          └──────────────┘          │              │
                                                    │              │
┌──────────────┐          ┌──────────────┐          │              │
│     RTC      │──────────>│Queue (RTC)   │──────────>│              │
└──────────────┘          └──────────────┘          └──────────────┘
                                                          │
                                                          │
┌──────────────┐          ┌──────────────┐                │
│Weather API   │──────────>│Queue (API)   │──────────>─────┘
└──────────────┘          └──────────────┘

┌──────────────┐
│  Buttons     │──────────> ISR ──────────> Screen State Change
└──────────────┘
```

# 4 Code Structure

## 4.1 Pin Definitions

```
#define SCREEN_CHANGE_BUTTON 18        // Button to cycle through screens
#define TIME_EDIT_ENABLE_BUTTON 32     // Button to edit time/date
#define DHTPIN 13                      // DHT11 data pin
#define PULSE_PIN 33                   // Pulse sensor analog input
```

## 4.2 Data Structures

### 4.2.1 Time Information

```
typedef struct{
  String date;    // Formatted date string
  String time;    // Formatted time string (HH:MM:SS)
  String AmPm;    // AM/PM indicator
}timeStrings;
```

### 4.2.2 DHT Sensor Data

```
typedef struct{
  float temp;     // Temperature in °C
  float rh;       // Relative humidity in %
}DHT_sensor_data;
```

### 4.2.3 Weather API Data

```
typedef struct{
  String description;     // Weather description (e.g., "clear sky")
  float tempFeelLike;     // Feels-like temperature (Kelvin)
  float humidity;         // Humidity percentage
  float windSpeed;        // Wind speed (m/s)
}openWeatherJSONParsed;
```

### 4.2.4 Screen Status

```
typedef struct{
  uint8_t screenCurrentIndex;         // Current screen (0-3)
  uint8_t currentBlinkingTimeField;   // Which time field blinks (0-5)
  uint8_t timeChange;                 // Time adjustment value
}ScreenStatus;
```

# 5 Task Implementations

## 5.1 Task 1: DHT Sensor Reading

**Purpose:** Reads temperature and humidity from DHT11 sensor every second.

**Stack Size:** 1150 bytes

**Key Points:**

- Uses `DHT_Unified` library for sensor communication
- Checks for `NaN` (Not a Number) to detect sensor failures
- Sends data through queue to display task
- Runs continuously with 1-second delay

```
void readDHT(void* parameters){
  DHT_sensor_data TempRHvalues;  // Local buffer for readings

  for(;;){  // Infinite loop (task never exits)
    vTaskDelay(1000 / portTICK_RATE_MS);  // Wait 1 second

    sensors_event_t event;
    dht.temperature().getEvent(&event);  // Get temperature reading

    if(isnan(event.temperature)){
      Serial.println("DHT11 FAILED TO READ TEMP");
    }else{
      Serial.print("DHT11 Temp = ");
      Serial.print(event.temperature);
      Serial.println("°C");
      TempRHvalues.temp = event.temperature;
    }

    dht.humidity().getEvent(&event);  // Get humidity reading

    if(isnan(event.relative_humidity)){
      Serial.println("DHT11 FAILED TO READ RELATIVE HUMIDITY");
    }else{
      Serial.print("DHT11 REL_HUMIDITY = ");
      Serial.print(event.relative_humidity);
      Serial.println("%");
      TempRHvalues.rh = event.relative_humidity;
    }

    // Send data to screen task via queue
    xQueueSend(screenDHTQueue_handle, &TempRHvalues, portMAX_DELAY);

    // Debug: Check remaining stack space
    Serial.print("Free Stack DHT: ");
    Serial.println(uxTaskGetStackHighWaterMark(readDHT_handle));
  }
}
```

## 5.2 Task 2: Pulse Sensor Reading

**Purpose:** Monitors analog pulse sensor and calculates heart rate (BPM).

**Stack Size:** 3000 bytes

**Algorithm:**

1. Read analog signal from pulse sensor
2. Detect peaks above threshold (2650)
3. Calculate time between peaks
4. Convert to BPM (beats per minute)
5. Average last 10 valid readings for stability

**Key Features:**

- Debouncing with 400ms minimum delay between peaks
- Validates BPM range (40-120) to filter noise
- Rolling average of 10 readings for smooth output

```
void readPulseSensor(void *parameters){
  uint32_t lastPeakTime = 0;
  uint16_t threshold = 2650;        // Analog threshold for peak detection
  uint32_t timeDelay = 400;         // Minimum time between peaks (ms)
  bool pulseOcurred = false;

  uint8_t READING_NUM = 10;
  uint8_t CURRENT_INDEX = 0;
  uint16_t lastPulses[READING_NUM] = {0,0,0,0,0,0,0,0,0,0};  // Circular buffer

  uint16_t signal;
  uint16_t BPM = 0;
  uint16_t tempBPM;

  for(;;){
    signal = analogRead(PULSE_PIN);  // Read sensor (0-4095 on ESP32)

    // Peak detected above threshold
    if(signal > threshold && !pulseOcurred){

      uint32_t currentTime = millis();

      // Check if enough time passed since last peak
      if(currentTime - lastPeakTime > timeDelay){

        // Calculate BPM: 60000ms per minute / time between beats
        tempBPM = 60000 / (currentTime - lastPeakTime);

        // Only accept reasonable heart rates
        if(tempBPM>40 && tempBPM<120){
          lastPulses[CURRENT_INDEX] = tempBPM;
          CURRENT_INDEX = (CURRENT_INDEX+1)%10;  // Circular buffer
        }

        lastPeakTime = currentTime;
        pulseOcurred = !pulseOcurred;  // Mark peak detected
      }

      // Calculate average of valid readings
      uint16_t sumBPM = 0;
      uint8_t validReading = 0;
```

```
      for(int i=0;i<=READING_NUM;i++){  // Note: Should be i<READING_NUM
        if(lastPulses[i]>0){
            sumBPM += lastPulses[i];
            validReading++;
        }
      }

      if(validReading>0){
        BPM = sumBPM/validReading;  // Average BPM
      }

      Serial.print("Current BPM = ");
      Serial.println(BPM);

      // Send to display task
      xQueueSend(screenPulseQueue_handle, &BPM, portMAX_DELAY);
    }

    // Reset when signal drops below threshold
    if(signal < (threshold-100) && pulseOcurred){
      pulseOcurred = !pulseOcurred;
    }

    vTaskDelay(10/portTICK_PERIOD_MS);  // Short delay (100 samples/sec)
  }
}
```

## 5.3 Task 3: Real-Time Clock Reading

**Purpose:** Reads system time and date every second from ESP32's internal RTC.

**Stack Size:** 3000 bytes

**Key Points:**

- Uses `ESP32Time` library for RTC access
- Formats time and date as strings
- Uses `xQueueOverwrite` instead of `xQueueSend` (always keeps latest time)

```
void readRTC(void *parameters){
  tm timeInfo;          // Standard C time structure
  timeStrings strTime;  // Our custom time strings

  for(;;){
    if(getLocalTime(&timeInfo)){  // Get time from system
      strTime.date = rtc.getDate(false);  // Get formatted date
      strTime.time = rtc.getTime();       // Get formatted time
      strTime.AmPm = rtc.getAmPm(true);   // Get AM/PM

      // Overwrite queue (only latest time matters)
      xQueueOverwrite(screenRTCQueue_handle, &strTime);

    }else{
      Serial.println("FAILED TO READTIME");
    }
    vTaskDelay(1000/portTICK_PERIOD_MS);  // Update every second
  }
}
```

## 5.4 Task 4: Weather API Fetching

**Purpose:** Fetches current weather data from OpenWeatherMap API every 5 seconds.

**Stack Size:** 4000 bytes

**Process:**

1. Make HTTP GET request to OpenWeather API
2. Parse JSON response
3. Extract weather description, temperature, humidity, wind speed
4. Send to display task via queue

```
void openWeatherGet(void* parameters){
  openWeatherJSONParsed weatherInfoBuffer;
  String tempJSON;
  JSONVar tempJSONVar;

  for(;;){
    httpClient.begin(openWeatherUrl);  // Initialize HTTP connection
    int httpCode = httpClient.GET();   // Make GET request

    if(httpCode>0){  // Successful connection
      Serial.println("---- HOLA! CONNECTED HTTP ----");
      tempJSON = httpClient.getString();      // Get response body
      tempJSONVar = JSON.parse(tempJSON);     // Parse JSON

      // Extract fields from JSON (e.g., weatherInfoBuffer.description)
      weatherInfoBuffer.description =
        tempJSONVar.stringify(tempJSONVar["weather"][0]["description"]);
      weatherInfoBuffer.tempFeelLike =
```

```
        atof(tempJSONVar.stringify(tempJSONVar["main"]["feels_like"]).c_str());
      weatherInfoBuffer.humidity =
        atof(tempJSONVar.stringify(tempJSONVar["main"]["humidity"]).c_str());
      weatherInfoBuffer.windSpeed =
        atof(tempJSONVar.stringify(tempJSONVar["wind"]["speed"]).c_str());

      // Send to display task
      xQueueSend(screenOpenWeather_handle, &weatherInfoBuffer, portMAX_DELAY);

      // Debug output
      Serial.print("Description = ");
      Serial.println(weatherInfoBuffer.description.substring(1,
        weatherInfoBuffer.description.length()-1));
      Serial.print("API TEMP = ");
      Serial.println(weatherInfoBuffer.tempFeelLike);

      httpClient.end();  // Close connection
      vTaskDelay(5000/portTICK_PERIOD_MS);  // Wait 5 seconds

    }else{
      Serial.println("HELL NAH");  // Connection failed
    }
    vTaskDelay(5000/portTICK_PERIOD_MS);
  }
}
```

## 5.5 Task 5: Screen Display Controller

**Purpose:** Updates OLED display based on current screen and sensor data.

**Stack Size:** 5000 bytes

**Screen Modes:**

- **Screen 0:** Time and Date display (with edit mode blinking)
- **Screen 1:** DHT11 Temperature and Humidity
- **Screen 2:** Heart Rate (BPM)
- **Screen 3:** Weather Information from API

**Key Features:**

- Cycles through screens with button press
- Implements blinking for time/date editing
- Receives data from all queues
- Updates every 400ms

```
void screenDisplay(void *parameters){
  uint8_t currentScreenIndex = 0;
  timeStrings tmInfoBuffer;
  DHT_sensor_data TempRHvaluesBuffer;
  uint16_t pulseReadingBuffer;
  openWeatherJSONParsed weatherInfoBuffer;
  String timeDateFrames[7];  // Stores different time/date format strings

  bool currentBlinkingState = false;  // For blinking effect

  for(;;){
    vTaskDelay(pdMS_TO_TICKS(400));  // Update every 400ms

    // Screen 0: Time and Date
    if(screenStatusCfx.screenCurrentIndex == 0){
      xQueueReceive(screenRTCQueue_handle, &tmInfoBuffer, pdMS_TO_TICKS(10));
      createTimeDateFrames(timeDateFrames, tmInfoBuffer.time, tmInfoBuffer.date);
      currentBlinkingState = !currentBlinkingState;  // Toggle blink

      // Blinking logic for time editing (hours or minutes)
      if((screenStatusCfx.currentBlinkingTimeField==1 ||
          screenStatusCfx.currentBlinkingTimeField==2) &&
          currentBlinkingState==true){

        int x = (screenStatusCfx.currentBlinkingTimeField==1) ? 1 : 2;

        screen.clearBuffer();
        screen.setFont(u8g2_font_helvB12_te);
        screen.drawStr(40,25, timeDateFrames[x].c_str());  // Show with dashes
        screen.setFont(u8g2_font_helvR08_te);
        screen.drawStr(20,50, timeDateFrames[3].c_str());
        screen.sendBuffer();
      }
      // Blinking for date fields
      else if (screenStatusCfx.currentBlinkingTimeField>=3 &&
               currentBlinkingState==true){
        screen.clearBuffer();
        screen.setFont(u8g2_font_helvB12_te);
        screen.drawStr(40,25, timeDateFrames[0].c_str());
        screen.setFont(u8g2_font_helvR08_te);
        screen.drawStr(20,50,
```

```
          timeDateFrames[screenStatusCfx.currentBlinkingTimeField+1].c_str());
        screen.sendBuffer();
      }
      // Normal display (no blinking)
      else{
        screen.clearBuffer();
        screen.setFont(u8g2_font_helvB12_te);
        screen.drawStr(40,25, timeDateFrames[0].c_str());
        screen.setFont(u8g2_font_helvR08_te);
        screen.drawStr(20,50, timeDateFrames[3].c_str());
        screen.sendBuffer();
      }
    }

    // Screen 1: DHT Sensor Data
    else if(screenStatusCfx.screenCurrentIndex == 1){
      xQueueReceive(screenDHTQueue_handle, &TempRHvaluesBuffer, pdMS_TO_TICKS(10));
      screen.clearBuffer();
      screen.setFont(u8g2_font_helvB10_te);
      screen.setCursor(20,25);
      screen.printf("Temp = %.2f ", TempRHvaluesBuffer.temp);
      screen.setCursor(30,50);
      screen.printf("RH = %.2f", TempRHvaluesBuffer.rh);
      screen.sendBuffer();
    }

    // Screen 2: Pulse Sensor (BPM)
    else if(screenStatusCfx.screenCurrentIndex == 2){
      xQueueReceive(screenPulseQueue_handle, &pulseReadingBuffer, pdMS_TO_TICKS(10));
      screen.clearBuffer();
      screen.setFont(u8g2_font_helvB12_te);
      screen.drawStr(40, 25, "BPM");
      screen.setCursor(50,50);
      screen.print(pulseReadingBuffer);
      screen.sendBuffer();
    }

    // Screen 3: Weather API Data
    else{
      xQueueReceive(screenOpenWeather_handle, &weatherInfoBuffer, pdMS_TO_TICKS(10));
      screen.clearBuffer();
      screen.setFont(u8g2_font_helvB08_tr);
      screen.drawStr(24,17, weatherInfoBuffer.description.substring(1,
        weatherInfoBuffer.description.length()-1).c_str());
      screen.setCursor(14,34);
      screen.print("Temp: ");
      screen.print(weatherInfoBuffer.tempFeelLike - 273.25);  // Kelvin to Celsius
      screen.setCursor(14,64);
      screen.print("RH(%): ");
      screen.print(weatherInfoBuffer.humidity);
      screen.setCursor(14,47);
      screen.print("wind speed: ");
      screen.print(weatherInfoBuffer.windSpeed);
      screen.sendBuffer();
    }
  }
}
```

# 6 Interrupt Service Routines (ISRs)

## 6.1 Screen Change Button ISR

**Purpose:** Cycles through the 4 display screens when button is pressed.

**Features:**

- Software debouncing (250ms)
- Modulo arithmetic to wrap around (0→1→2→3→0)
- Runs in IRAM for fast execution

```
void IRAM_ATTR screenChangeButtonISR(){
  unsigned long currentTime = millis();

  // Debounce: ignore if pressed within 250ms of last press
  if(currentTime - lastScreenChangeTime > DEBOUNCE_TIME){
    // Cycle through screens: (0+1)%4=1, (1+1)%4=2, ..., (3+1)%4=0
    screenStatusCfx.screenCurrentIndex =
      (screenStatusCfx.screenCurrentIndex + 1)%4;
    lastScreenChangeTime = currentTime;
  }
}
```

## 6.2 Time/Date Edit Enable Button ISR

**Purpose:** Cycles through time/date fields for editing (blinking indicator).

**Blinking Fields:**

- 0: No blinking
- 1: Hours blink
- 2: Minutes blink
- 3: Day blinks
- 4: Month blinks
- 5: Year blinks

```
void IRAM_ATTR screenTimeDateEditEnableButtonISR(){
  unsigned long currentTime = millis();

  if(currentTime - lastTimeEditEnablePressed > DEBOUNCE_TIME){
    lastTimeEditEnablePressed = currentTime;
    // Cycle through 6 states (0-5)
    screenStatusCfx.currentBlinkingTimeField =
      (screenStatusCfx.currentBlinkingTimeField+1)%6;
  }
}
```

# 7 Setup and Initialization

The `setup()` function initializes all hardware and creates FreeRTOS tasks.

## 7.1 Initialization Sequence

1. **Serial Communication** - 115200 baud for debugging
2. **DHT Sensor** - Initialize DHT11
3. **GPIO Pins** - Configure button inputs with pull-ups
4. **Interrupts** - Attach ISR functions to buttons
5. **Queues** - Create communication queues between tasks
6. **WiFi Connection** - Connect to WiFi network
7. **NTP Time Sync** - Get accurate time from internet
8. **I2C & Display** - Initialize OLED screen
9. **Task Creation** - Create and pin all 5 tasks to CPU cores

```
void setup(){
  Serial.begin(115200);
  delay(1000);

  dht.begin();  // Initialize DHT11 sensor

  // Configure button pins as inputs with internal pull-up resistors
  pinMode(SCREEN_CHANGE_BUTTON, INPUT_PULLUP);
  pinMode(TIME_EDIT_ENABLE_BUTTON, INPUT_PULLUP);

  // Attach interrupt handlers (trigger on FALLING edge = button press)
  attachInterrupt(digitalPinToInterrupt(SCREEN_CHANGE_BUTTON),
    screenChangeButtonISR, FALLING);
  attachInterrupt(digitalPinToInterrupt(TIME_EDIT_ENABLE_BUTTON),
    screenTimeDateEditEnableButtonISR, FALLING);

  // Create binary semaphore (not used in current implementation)
  screenDisplaySemaphore_handle = xSemaphoreCreateBinary();

  // Create queues for inter-task communication
  screenRTCQueue_handle = xQueueCreate(1, sizeof(timeStrings));
  screenDHTQueue_handle = xQueueCreate(SCREEN_DHT_QUEUE_SIZE, sizeof(DHT_sensor_data));
  screenPulseQueue_handle = xQueueCreate(SCREEN_PULSE_QUEUE_SIZE, sizeof(uint16_t));
  screenOpenWeather_handle = xQueueCreate(SCREEN_WEATHER_API_QUEUE_SIZE,
    sizeof(openWeatherJSONParsed));

  // Connect to WiFi
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  Serial.printf("Connecting to %s", WIFI_SSID);

  while(WiFi.status() != WL_CONNECTED){
    Serial.print(".");
    delay(250);
  }

  // Configure NTP time synchronization
  configTime(gmOffset, dayLightSaving, ntpServer1, ntpServer2);

  // Initialize I2C and OLED display
  Wire.begin();
  screen.begin();
```

```
  // Create all tasks and pin them to Core 1
  xTaskCreatePinnedToCore(readDHT, "DHT SENSOR READING TASK",
    1150, NULL, 1, &readDHT_handle, 1);

  xTaskCreatePinnedToCore(readPulseSensor, "READ PULSE SENSOR TASK",
    3000, NULL, 1, &readPulseSensor_handle, 1);

  xTaskCreatePinnedToCore(readRTC, "READ INTERNAL RTC TASK",
    3000, NULL, 1, &readRTC_handle, 1);

  xTaskCreatePinnedToCore(openWeatherGet, "OpenWeatherAPI Task",
    4000, NULL, 1, NULL, 1);

  xTaskCreatePinnedToCore(screenDisplay, "OLED DISPLAY TASK",
    5000, NULL, 1, NULL, 1);
}

void loop(){
  // Empty - all work done in FreeRTOS tasks
}
```

# 8 Helper Functions

## 8.1 Time/Date Frame Creator

This function creates different formatted versions of time and date strings for the blinking edit mode.

```
void createTimeDateFrames(String arr[], String time, String date,
                          int size = 7, bool mode=true){
  if(mode){
    arr[0] = time.substring(0,5);  // "HH:MM" (without seconds)
  }else{
    arr[0] = time;                 // Full time with seconds
  }

  // Different formats with dashes to show which part is being edited
  arr[1] = time.substring(0,3) + "--";           // "HH:--" (minutes hidden)
  arr[2] = "--" + time.substring(2,5);           // "--:MM" (hours hidden)
  arr[3] = date;                                 // Full date
  arr[4] = date.substring(0,9) + "--" + date.substring(11,date.length());  // Day hidden
  arr[5] = date.substring(0,5) + "---" + date.substring(8,date.length());  // Month hidden
  arr[6] = date.substring(0,12) + "----";        // Year hidden
}
```

**Usage:** When editing time, the function creates strings where the editable field is replaced with dashes. The display task toggles between showing the actual value and the dashed version to create a blinking effect.

# 9 Memory and Performance Considerations

## 9.1 Stack Sizes Explained

Each task is allocated a specific amount of stack memory:

| Task | Stack (bytes) | Justification |
|---|---|---|
| `readDHT` | 1150 | Small: Simple sensor reading + string handling |
| `readPulseSensor` | 3000 | Medium: Array operations + calculations |
| `readRTC` | 3000 | Medium: String formatting operations |
| `openWeatherGet` | 4000 | Large: HTTP client + JSON parsing |
| `screenDisplay` | 5000 | Largest: Display buffer + string operations |

**Note:** The code includes stack monitoring:

```
uxTaskGetStackHighWaterMark(readDHT_handle);
```

This returns the minimum free stack space ever reached (lower = closer to overflow).

## 9.2 Queue Sizes

| Queue | Size | Reason |
|---|---|---|
| RTC Queue | 1 | Only latest time matters (uses overwrite) |
| DHT Queue | 5 | Buffer several readings |
| Pulse Queue | 10 | Smooth BPM display with history |
| Weather Queue | 1 | Only latest weather data needed |

## 9.3 Core Pinning Strategy

All tasks are pinned to **Core 1**, leaving **Core 0** for:

- WiFi/Bluetooth radio tasks (handled by ESP-IDF)
- Arduino core functions
- System tasks

This prevents conflicts between user tasks and system operations.

# 10 System Behavior Summary

## 10.1 Normal Operation Flow

1. System boots and connects to WiFi
2. NTP syncs time from the internet
3. All 5 tasks start running concurrently:
   - DHT reads temp/humidity every 1s
   - Pulse sensor monitors continuously (10ms sampling)
   - RTC updates time every 1s
   - Weather API fetches every 5s
   - Display updates every 400ms
4. The display shows Screen 0 (Time/Date) by default
5. Press the Screen Change button → cycle through screens
6. Press Time Edit button → cycle through blinking fields

## 10.2 Button Interactions

**Screen Change Button (Pin 18):**

- Short press → Next screen (0→1→2→3→0